

ENGINEERING REBOOT 2026

WEEK 08 MASTER WORKBOOK

Object-Oriented Design: Inheritance, Polymorphism & Extensible Systems

DAYS 36-40 | MASTER WORKBOOK v1.0

Phase II - Modern Software Engineering Foundations

Publication Edition - August 2026

1. Week Mission

Week 08 extends the object-oriented foundations established in Week 07. The emphasis moves from constructing individual classes and managing object relationships to designing related object types that share behavior, specialize behavior, and cooperate through clear responsibilities.

The week introduces inheritance, method overriding, `super ( )`, and polymorphism, then deliberately compares inheritance with composition. The objective is not to build deep class hierarchies; it is to develop sound judgment about when these mechanisms make a design clearer.

Week 08 Learning Objectives

- Explain base classes, subclasses, inherited state, and inherited behavior.
- Recognize legitimate **is-a** relationships and distinguish them from **has-a** relationships.
- Override inherited methods and extend parent initialization with `super ( )`.
- Demonstrate polymorphism by processing different object types through common behavior.
- Choose between inheritance and composition based on the modeled relationship.
- Integrate inheritance, polymorphism, composition, and encapsulation in a small multi-class model.
- Keep solutions proportional to the learning objective and the one-hour session target.

2. Weekly Roadmap

Day	Topic	Primary Outcome
36	Inheritance Fundamentals	Model meaningful specialization with base and derived classes.
37	Overriding & <code>super()</code>	Specialize behavior without duplicating parent initialization.
38	Polymorphism	Process mixed object types through a common operation.
39	Composition vs. Inheritance	Select the correct relationship and justify the decision.
40	Integration Project	Build an Employee Payroll Model combining the week's concepts.

3. Day 36 - Inheritance Fundamentals

Mission. Understand inheritance as a mechanism for representing legitimate specialization relationships between classes.

Core Concepts

- Base and derived classes
- Inherited attributes and methods
- Subclass-specific behavior
- The is-a relationship
- Python inheritance syntax

Build / Mini-Project

Employee Hierarchy - Implement a compact Employee base class with SalariedEmployee and HourlyEmployee subclasses. Demonstrate what is inherited before adding subclass-specific behavior.

Definition of Done

- Define and instantiate a base class and subclasses.
- Demonstrate inherited behavior.
- Add subclass-specific behavior.
- Explain why the modeled relationship qualifies as inheritance.

4. Day 37 - Method Overriding and super()

Mission. Learn how subclasses specialize inherited behavior while retaining appropriate parent functionality.

Core Concepts

- Method overriding
- Constructor specialization
- super()
- Shared versus specialized behavior
- Avoiding duplicated initialization

Build / Mini-Project

Specialized Employee Behavior - Extend the Day 36 model. Introduce subclass-specific state, override selected behavior, and use super() where parent initialization remains relevant.

Definition of Done

- Override an inherited method.
- Use super() to extend parent initialization.
- Explain which implementation is invoked.
- Avoid unnecessary duplication.

5. Day 38 - Polymorphism

Mission. Use common behavior across different object types without forcing consuming code to understand every concrete implementation.

Core Concepts

- Common behavioral contracts
- Dynamic method dispatch
- Mixed subclass collections
- Behavior-driven processing
- Reducing type-specific branching

Build / Mini-Project

Polymorphic Payroll Processing - Process SalariedEmployee and HourlyEmployee objects through a shared calculate_pay() operation. Contrast this with type-driven branching.

Definition of Done

- Process multiple subclass types together.
- Invoke the same operation and obtain type-specific behavior.
- Avoid subclass checks in the normal processing path.
- Explain the engineering value of polymorphism.

6. Day 39 - Composition vs. Inheritance

Mission. Develop judgment about when a relationship should be modeled through inheritance and when it should use composition.

Core Concepts

- is-a versus has-a
- Composition
- Inheritance
- Delegation
- Responsibility placement

Build / Mini-Project

Employee Management Model - Evaluate several object relationships, justify the design choice, then combine the Employee hierarchy with a class that contains or manages employee objects.

Definition of Done

- Distinguish is-a from has-a relationships.
- Select inheritance or composition appropriately.
- Explain the design decision.

- Combine both relationships in a small object model.

7. Day 40 - Week Integration Project

Mission. Integrate the Week 08 concepts into a coherent but deliberately bounded object-oriented model.

Core Concepts

- Inheritance
- Overriding
- `super()`
- Polymorphism
- Composition
- Encapsulation

Build / Mini-Project

Employee Payroll Model - Build a small model with an Employee base class, at least two concrete employee types, and a Payroll object that contains employees and processes them polymorphically.

Definition of Done

- Use one meaningful base class and at least two derived classes.
- Demonstrate inherited and overridden behavior.
- Use `super()` appropriately.
- Process a mixed collection polymorphically.
- Use at least one meaningful composition relationship.
- Avoid unnecessary subclass-specific branching.

8. Week 08 Engineering Principles

Inheritance represents meaning. Do not introduce inheritance solely to eliminate duplicate lines of code.

Prefer behavior over type inspection. Let participating objects determine how a shared operation behaves.

Composition remains a first-class design tool. Use composition when one object has, contains, manages, or uses another.

Keep hierarchies shallow. Week 08 exercises should use small, understandable hierarchies.

Demonstrate the concept, then stop. Additional sophistication is not automatically additional learning.

9. Scope Guardrails

- Databases or persistence
- Abstract base classes and protocols
- Design-pattern implementations
- Complex exception hierarchies
- Elaborate command-line interfaces
- Production payroll rules
- Deep inheritance trees

SESSION DISCIPLINE

Target approximately one focused hour per day. Concept review, exercises, implementation, evidence, Notes.md, Journal.md, and Git operations should fit within that working envelope. If the mini-project itself consumes most of the session, reduce scope rather than compressing the curriculum.

10. Week Completion Criteria

- Days 36-40 are completed and archived according to Engineering Reboot standards.
- Independently recognize and implement an appropriate inheritance relationship.
- Override behavior and use `super()` appropriately.
- Explain and demonstrate polymorphism.
- Distinguish inheritance from composition and justify the choice.
- Construct a small application combining these techniques without unnecessary complexity.